



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

**FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y
DISEÑO DIGITAL**

CARRERA DE SOFTWARE

APLICACIONES WEB – SOFT-R – A – [5TO NIVEL] | CORTE 1

TEMA:

GA – Práctica en clase: Proyecto ASP.NET Core MVC

ESTUDIANTE:

PANAMA MURILLO MOISES ANTONIO

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web 5to Nivel

Contents

1	Introduccion	2
2	Entorno y herramientas	2
3	Paso 1 — Creacion del proyecto	2
3.1	Comandos ejecutados	2
3.2	Descripcion	2
3.3	Evidencia en terminal	2
3.4	Estructura de carpetas generada	4
4	Paso 2 — Modelo Prestamo y PrestamoController	4
4.1	Codigo del modelo	4
4.2	Archivo <code>Prestamo.cs</code> en el editor	5
4.3	Codigo del controlador (Index y Create)	5
4.4	Archivo <code>PrestamoController.cs</code> en el editor	7
5	Paso 3 — Vistas Razor	7
5.1	Codigo de la vista <code>Index</code>	7
5.2	Vista <code>Index</code> en el navegador	8
5.3	Codigo de la vista <code>Create</code>	8
5.4	Formulario <code>Create</code> en el navegador	9
6	Paso 4 — Accion Delete con confirmacion, POST y CSRF	10
6.1	Codigo del controlador (Delete)	10
6.2	Vista de confirmacion en el navegador	11
7	Paso 5 — Verificacion de la validacion (server-side y cliente)	11
7.1	Errores de validacion en el formulario	11
7.2	Prestamo creado exitosamente	12
8	Repositorio GitHub	12
8.1	Flujo de trabajo utilizado	12
8.2	Historial de commits	13
8.3	Enlace al repositorio	13
9	Conclusiones	13

1 Introduccion

Este proyecto implementa el modulo de **prestamos** del sistema **GestBiblioteca** utilizando **ASP.NET Core MVC**. A traves de la gestion de prestamos de libros se aplicaron los conceptos de validacion de datos con *Data Annotations*, proteccion de formularios con **AntiForgeryToken**, eliminacion segura mediante confirmacion POST con proteccion CSRF, y manejo de rutas mediante controladores y vistas Razor.

2 Entorno y herramientas

- **IDE:** IntelliJ IDEA (terminal integrada para compilar y ejecutar)
- **SDK:** .NET 10
- **Sistema operativo:** Windows
- **Control de versiones:** Git + GitHub
- **Repositorio:** <https://github.com/MoisesPanama/GestBiblioteca>
- **Puerto local:** <http://localhost:5248/Prestamo>

3 Paso 1 — Creacion del proyecto

3.1 Comandos ejecutados

```
mkdir GestBiblioteca
cd GestBiblioteca
dotnet new mvc -n GestBibliotecaApp
cd GestBibliotecaApp
dotnet run
```

3.2 Descripcion

Se ejecuto el comando `dotnet new mvc -n GestBibliotecaApp` en la terminal integrada de IntelliJ IDEA, generando automaticamente la estructura base del proyecto con las carpetas **Controllers**, **Models**, **Views**, **wwwroot** y los archivos **Program.cs** y **GestBibliotecaApp.csproj**. Al ejecutar `dotnet run` el servidor quedo escuchando en <http://localhost:5248>, confirmando que el proyecto se creo sin errores.

3.3 Evidencia en terminal

Se ejecutaron los comandos en la terminal integrada de IntelliJ IDEA, generando la estructura base y confirmando que el servidor levanto correctamente.

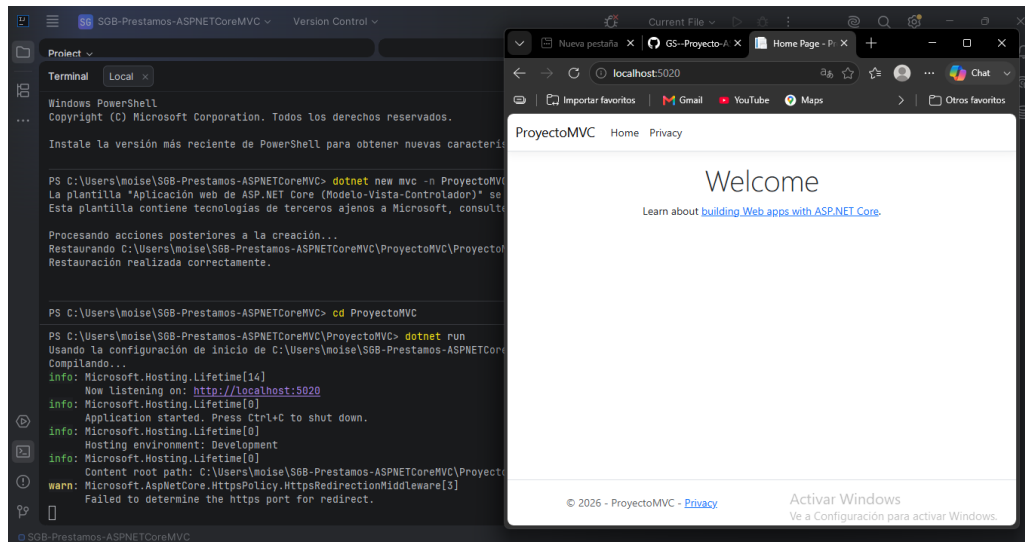


Figure 1: Terminal mostrando `dotnet new mvc -n GestBibliotecaApp` y `dotnet run` ejecutados correctamente.

3.4 Estructura de carpetas generada

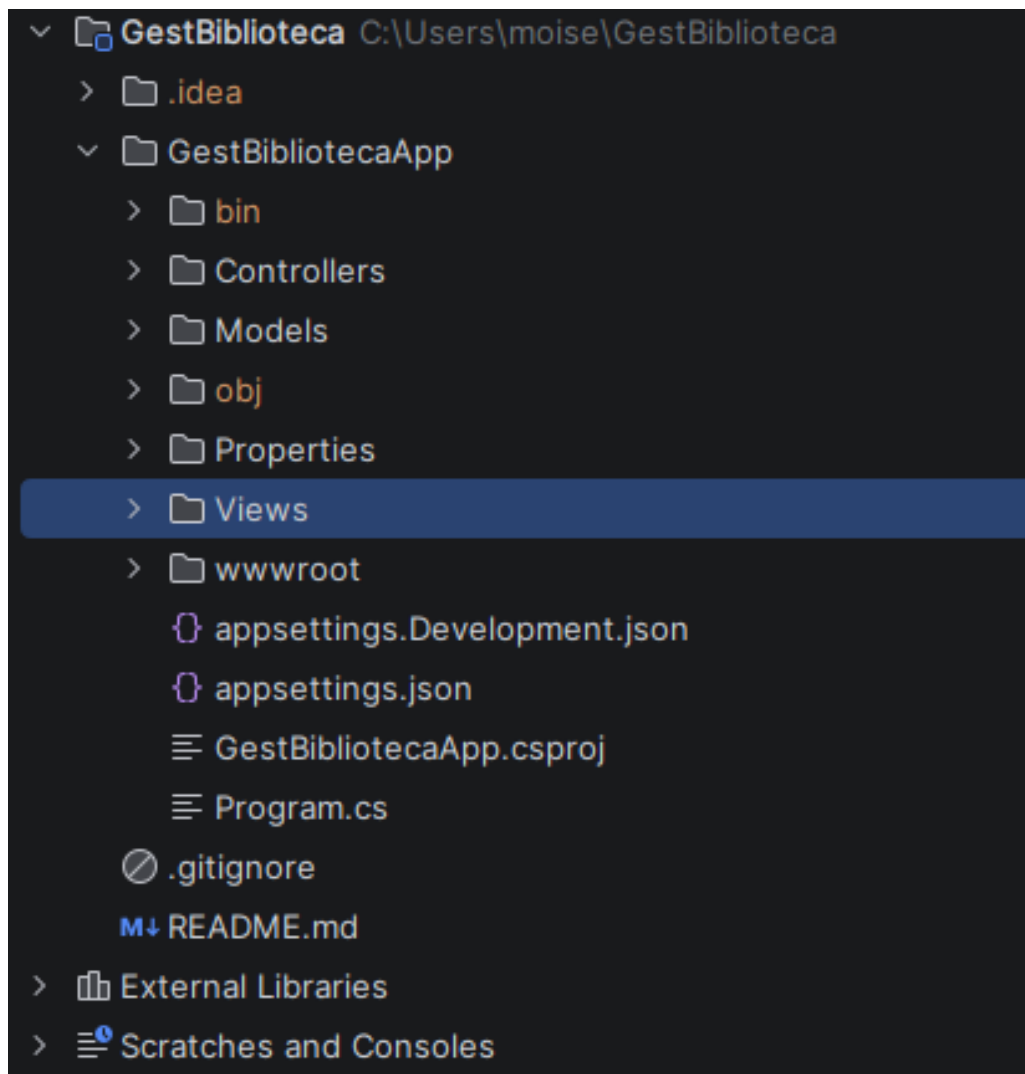


Figure 2: Estructura de carpetas del proyecto generado visible en IntelliJ IDEA.

4 Paso 2 — Modelo Prestamo y PrestamoController

4.1 Código del modelo

```
using System.ComponentModel.DataAnnotations;

namespace GestBibliotecaApp.Models
{
    public class Prestamo
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre del estudiante es obligatorio")]
        [StringLength(100, ErrorMessage = "Maximo 100 caracteres")]
        [Display(Name = "Estudiante")]
        public string? Estudiante { get; set; }

        [Required(ErrorMessage = "El titulo del libro es obligatorio")]
        [StringLength(150, ErrorMessage = "Maximo 150 caracteres")]
        [Display(Name = "Libro")]
    }
}
```

```

public string? Libro { get; set; }

[Required(ErrorMessage = "La fecha de prestamo es obligatoria")]
[DataType(DataType.Date)]
[Display(Name = "Fecha de Prestamo")]
public DateTime FechaPrestamo { get; set; }

[Required(ErrorMessage = "La fecha de devolucion es obligatoria")]
[DataType(DataType.Date)]
[Display(Name = "Fecha de Devolucion Estimada")]
public DateTime FechaDevolucion { get; set; }

[Required(ErrorMessage = "El estado es obligatorio")]
[RegularExpression("Activo|Devuelto|Vencido|Renovado",
    ErrorMessage = "Use: Activo, Devuelto, Vencido o Renovado")]
[Display(Name = "Estado")]
public string? Estado { get; set; }
}
}

```

4.2 Archivo Prestamo.cs en el editor

Se creo el archivo Prestamo.cs dentro de la carpeta Models con las Data Annotations correspondientes a cada campo del modelo.

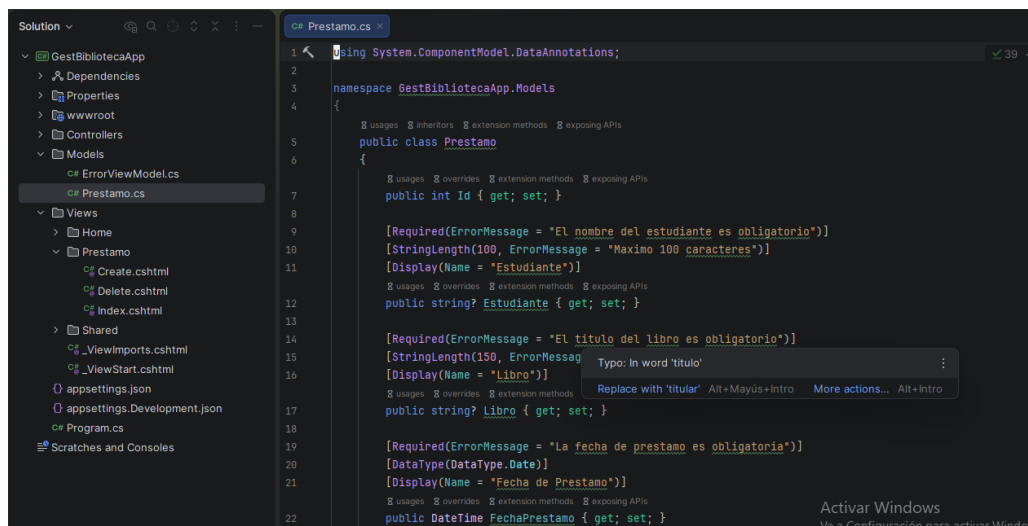


Figure 3: Archivo Prestamo.cs abierto en IntelliJ IDEA.

4.3 Codigo del controlador (Index y Create)

```

// GestBiblioteca - Modulo de Prestamos
// Extension: Delete con confirmacion POST y proteccion CSRF
using Microsoft.AspNetCore.Mvc;
using GestBibliotecaApp.Models;

namespace GestBibliotecaApp.Controllers
{
    public class PrestamoController : Controller
    {
        private static List<Prestamo> _prestamos = new List<Prestamo>();

        // GET: /Prestamo
        public IActionResult Index()

```

```
{
    return View(_prestamos);
}

// GET: /Prestamo/Create
public IActionResult Create()
{
    return View();
}

// POST: /Prestamo/Create
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Prestamo prestamo)
{
    if (ModelState.IsValid)
    {
        prestamo.Id = _prestamos.Count + 1;
        _prestamos.Add(prestamo);
        return RedirectToAction("Index");
    }
    return View(prestamo);
}

// GET: /Prestamo/Delete/5 (solo muestra confirmacion, nunca borra)
public IActionResult Delete(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo == null)
    {
        return NotFound();
    }
    return View(prestamo);
}

// POST: /Prestamo/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo != null)
    {
        _prestamos.Remove(prestamo);
    }
    return RedirectToAction("Index");
}
}
```

4.4 Archivo PrestamoController.cs en el editor

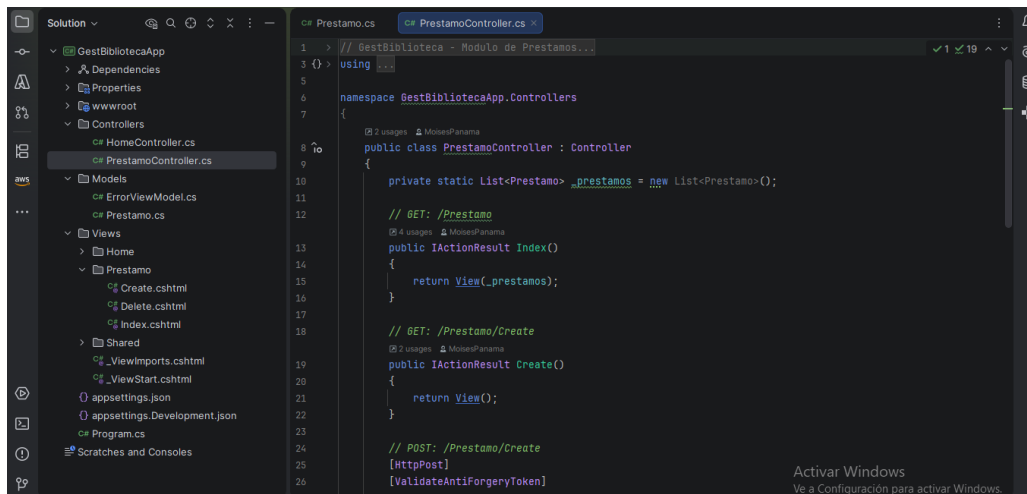


Figure 4: Archivo PrestamoController.cs con las acciones Index y Create en IntelliJ IDEA.

5 Paso 3 — Vistas Razor

5.1 Código de la vista Index

```
@* Vista Index - Lista de Prestamos *@
@model IEnumerable<GestBibliotecaApp.Models.Prestamo>

@{
    ViewData["Title"] = "Prestamos";
}

<div class="container mt-4">
    <h2 class="mb-3">Gestion de Prestamos</h2>

    <a asp-action="Create" class="btn btn-primary mb-3">+ Registrar Prestamo</a>

    @if (!Model.Any())
    {
        <div class="alert alert-info">No hay prestamos registrados aun.</div>
    }
    else
    {
        <table class="table table-bordered table-hover">
            <thead class="table-dark">
                <tr>
                    <th>#</th>
                    <th>Estudiante</th>
                    <th>Libro</th>
                    <th>F. Prestamo</th>
                    <th>F. Devolucion</th>
                    <th>Estado</th>
                    <th>Acciones</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var p in Model)
                {
                    <tr>
                        <td>@p.Id</td>
```

```

        <td>@p.Estudiante</td>
        <td>@p.Libro</td>
        <td>@p.FechaPrestamo.ToShortDateString()</td>
        <td>@p.FechaDevolucion.ToShortDateString()</td>
        <td>
            <span class="badge @(p.Estado == "Activo" ? "bg-success" :
                p.Estado == "Vencido" ? "bg-danger" :
                p.Estado == "Renovado" ? "bg-warning text-dark" :
                "bg-secondary")">
                @p.Estado
            </span>
        </td>
        <td>
            <a asp-action="Delete" asp-route-id="@p.Id"
                class="btn btn-danger btn-sm">Eliminar</a>
        </td>
    </tr>
}
</tbody>
</table>
}
</div>

```

5.2 Vista Index en el navegador

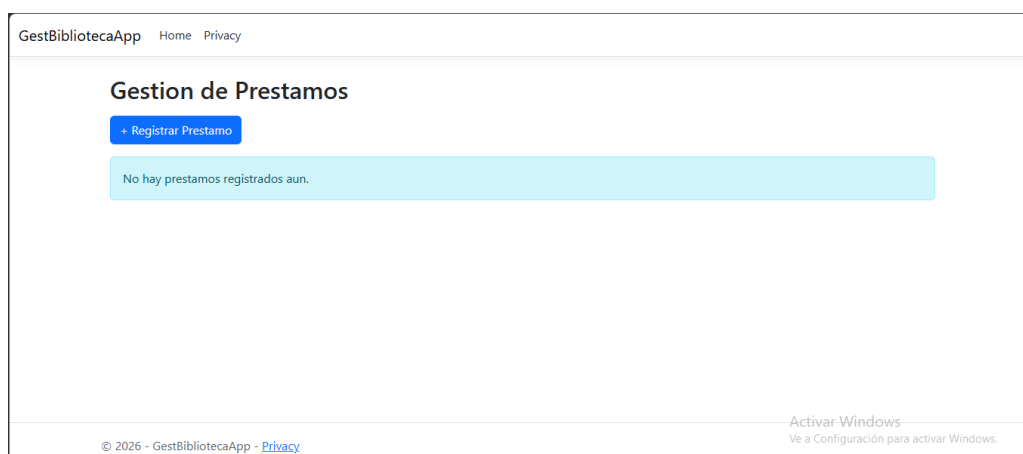


Figure 5: Vista Index en el navegador mostrando la tabla de prestamos.

5.3 Codigo de la vista Create

```

@* Vista Create - Registrar Prestamo - Validacion activada *@
@model GestBibliotecaApp.Models.Prestamo

@{
    ViewData["Title"] = "Registrar Prestamo";
}

<div class="container mt-4">
    <h2 class="mb-3">Registrar Prestamo</h2>

    <form asp-action="Create" method="post">
        @Html.AntiForgeryToken()

        <div class="mb-3">
            <label asp-for="Estudiante" class="form-label"></label>

```

```

        <input asp-for="Estudiante" class="form-control"
            placeholder="Nombre completo del estudiante" />
        <span asp-validation-for="Estudiante" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Libro" class="form-label"></label>
        <input asp-for="Libro" class="form-control"
            placeholder="Titulo del libro" />
        <span asp-validation-for="Libro" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="FechaPrestamo" class="form-label"></label>
        <input asp-for="FechaPrestamo" class="form-control" type="date" />
        <span asp-validation-for="FechaPrestamo" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="FechaDevolucion" class="form-label"></label>
        <input asp-for="FechaDevolucion" class="form-control" type="date" />
        <span asp-validation-for="FechaDevolucion" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Estado" class="form-label"></label>
        <select asp-for="Estado" class="form-control">
            <option value="">-- Seleccione un estado --</option>
            <option value="Activo">Activo</option>
            <option value="Devuelto">Devuelto</option>
            <option value="Vencido">Vencido</option>
            <option value="Renovado">Renovado</option>
        </select>
        <span asp-validation-for="Estado" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-success">Guardar</button>
    <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
</form>
</div>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}

```

5.4 Formulario Create en el navegador

El formulario incluye `@@Html.AntiForgeryToken()` para protegerse contra ataques CSRF. Al renderizar `_ValidationScriptsPartial` se habilita la **validacion cliente** (jQuery Unobtrusive Validation).

Figure 6: Formulario *Registrar Prestamo* en el navegador.

6 Paso 4 — Accion Delete con confirmacion, POST y CSRF

6.1Codigo del controlador (Delete)

```
// GET: /Prestamo/Delete/5 (solo muestra confirmacion, nunca borra)
public IActionResult Delete(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo == null)
    {
        return NotFound();
    }
    return View(prestamo);
}

// POST: /Prestamo/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p => p.Id == id);
    if (prestamo != null)
    {
        _prestamos.Remove(prestamo);
    }
    return RedirectToAction("Index");
}
```

Se implemento un `Delete` GET que solo muestra una vista de confirmacion (nunca borra datos en una peticion GET), y un `Delete` POST (`DeleteConfirmed`) protegido con `[ValidateAntiForgeryToken]` que ejecuta la eliminacion real.

```
// GET: /Prestamo/Delete/5 (solo muestra confirmacion, nunca borra)
[HttpGet]
public IActionResult Delete(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p : Prestamo => p.Id == id);
    if (prestamo == null)
    {
        return NotFound();
    }
    return View(prestamo);
}

// POST: /Prestamo/Delete/5 (borra real, protegido con CSRF)
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var prestamo = _prestamos.FirstOrDefault(p : Prestamo => p.Id == id);
    if (prestamo != null)
    {
        _prestamos.Remove(prestamo);
    }
}
```

Figure 7: Acciones Delete (GET) y DeleteConfirmed (POST + CSRF) en el controlador.

6.2 Vista de confirmacion en el navegador

GestBibliotecaApp Home Privacy

Eliminar Prestamo

Atencion: Esta accion no se puede deshacer. Confirma que deseas eliminar este registro.

Estudiante	Moisés Antonio Panamá Murillo
Libro	Harry Potter
Fecha de Prestamo	18/06/2026
Fecha de Devolucion	18/07/2026
Estado	Activo

[Confirmar eliminacion](#) [Cancelar](#)

© 2026 - GestBibliotecaApp - [Privacy](#)

Activar Windows
Ve a Configuración para activar Windows.

Figure 8: Vista de confirmacion antes de eliminar un prestamo.

7 Paso 5 — Verificacion de la validacion (server-side y cliente)

7.1 Errores de validacion en el formulario

Al enviar el formulario vacio o con datos invalidos, primero actua la **validacion cliente** (JavaScript) y, si esta se evita, la **validacion server-side**: el servidor evalua `ModelState.IsValid`; si es `false`, la accion `Create` regresa la misma vista mostrando los mensajes de error definidos en las Data Annotations, sin guardar el registro.

Registrar Prestamo

Estudiante

 El nombre del estudiante es obligatorio

Libro

 El título del libro es obligatorio

Fecha de Prestamo

 La fecha de prestamo es obligatoria

Fecha de Devolucion Estimada

 La fecha de devolucion es obligatoria

Estado

 El estado es obligatorio

Figure 9: Mensajes de error de validacion server-side.

7.2 Prestamo creado exitosamente

GestBibliotecaApp Home Privacy

Gestion de Prestamos

[+ Registrar Prestamo](#)

#	Estudiante	Libro	F. Prestamo	F. Devolucion	Estado	Acciones
1	Moisés Antonio Panamá Murillo	Harry Potter	18/06/2026	18/07/2026	Activo	Eliminar

© 2026 - GestBibliotecaApp - [Privacy](#)

Figure 10: Vista Index actualizada, mostrando el prestamo recién registrado.

8 Repositorio GitHub

8.1 Flujo de trabajo utilizado

1. **Version base en main:** proyecto, modelo Prestamo y acciones Index/Create.

```
git init
git add .
git commit -m "Paso 1: Proyecto MVC GestBiblioteca creado y probado"
git branch -M main
git remote add origin https://github.com/MoisesPanama/GestBiblioteca.git
git push -u origin main
```

2. **Rama de extension:** accion Delete con confirmacion, CSRF y validacion cliente.

```
git checkout -b feature/extension-delete-csrf
git add .
git commit -m "Paso 4: Accion Delete con confirmacion POST y CSRF"
git push origin feature/extension-delete-csrf
```

Luego: *Compare and pull request* → *Create pull request* → *Merge pull request* hacia **main**.

8.2 Historial de commits

#	Rama	Mensaje
1	main	Paso 1: Proyecto MVC GestBiblioteca creado y probado
2	main	Paso 2: Modelo Prestamo con Data Annotations y PrestamoController
3	main	Paso 3: Vistas Razor Index Create y Delete con AntiForgeryToken
4	feature/extension-delete-csrf	Paso 4: Accion Delete con confirmacion POST y CSRF
5	main	Paso 5: Validacion server-side verificada y proyecto finalizado

8.3 Enlace al repositorio

<https://github.com/MoisesPanama/GestBiblioteca>

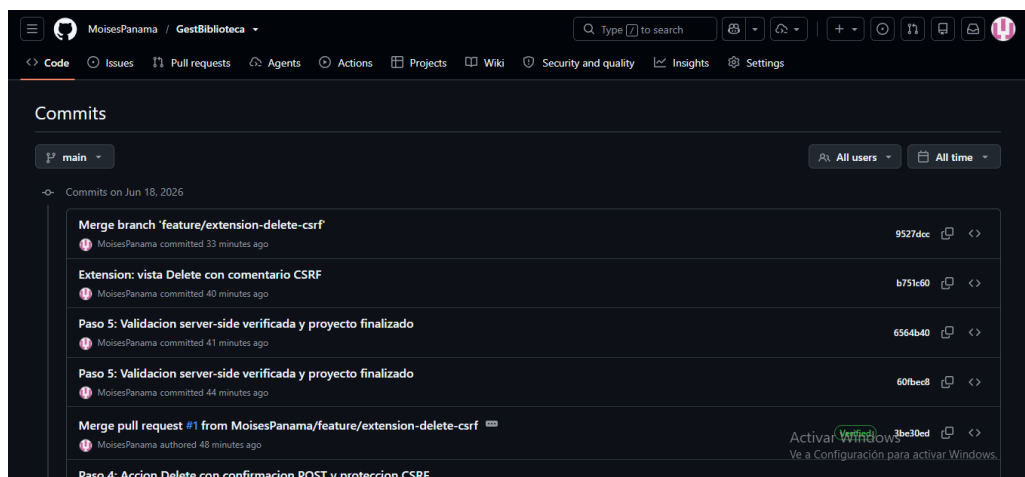


Figure 11: Repositorio en GitHub con el historial de commits y el Pull Request.

9 Conclusiones

El desarrollo de **GestBiblioteca** permitio comprender de forma practica el patron MVC en ASP.NET Core: el **modelo** centraliza las reglas de validacion mediante Data Annotations, el **controlador** orquesta el flujo entre las peticiones HTTP y los datos, y la **vista** Razor se encarga de la presentacion con Tag Helpers. La accion **Delete** con confirmacion reforzo la importancia de no ejecutar operaciones destructivas en peticiones GET, y el uso combinado de **AntiForgeryToken** con validacion cliente y server-side mostro como ASP.NET Core MVC protege la integridad de los datos en multiples capas. El flujo de ramas y Pull Request en GitHub permitio practicar un control de versiones cercano al de un proyecto de equipo real.